

# Paralelismo a nível de instruções (ILP)

João Canas Ferreira / António José Araújo

Março 2025

# Limites da execução paralela de instruções

# Execução paralela de instruções

- Execução de instruções “em-ordem” e em *pipeline*
  - CPI ideal de 1
  - O desempenho vem reduzido por:
    - Conflitos estruturais
      - Mitigado por memórias cache separadas (*split*)
    - Conflitos devidos a dependências de dados
      - Mitigado por atalhos (*forwarding*)
    - Conflitos devidos a dependências de controle
      - Mitigado por predição de saltos
- Para aumentar o número de instruções executadas em paralelo:
  - Emitir (iniciar) mais que uma instrução por ciclo!
  - Contudo: **conflitos serão, previsivelmente, ainda mais frequentes**
  - Como reduzir **ainda mais** o impacto dos conflitos causados por dependências de dados e controle? E vale a pena?

# Limites do paralelismo ao nível de instruções

- Quanto paralelismo está presente em programas convencionais?
- O que é necessário para aproveitar esse paralelismo latente?

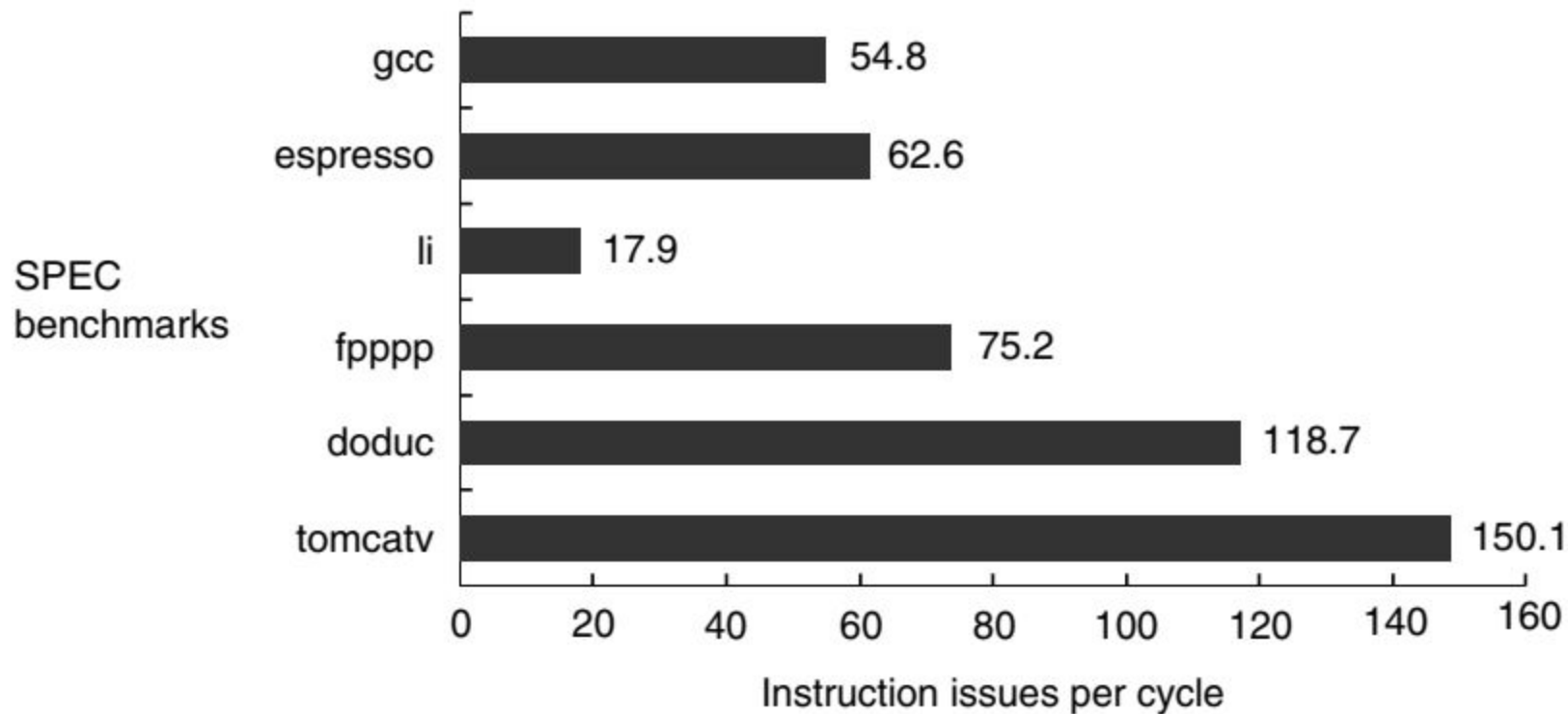
Estudo do desempenho de um **processador perfeito**:

- Número infinito de registos
- Predição exata de saltos condicionais
- Conhecimento perfeito de endereços de memória
  - Por exemplo, saber se as duas instruções seguintes **se referem ou não** à mesma posição de memória.

```
sw    t4, 200(t5)
```

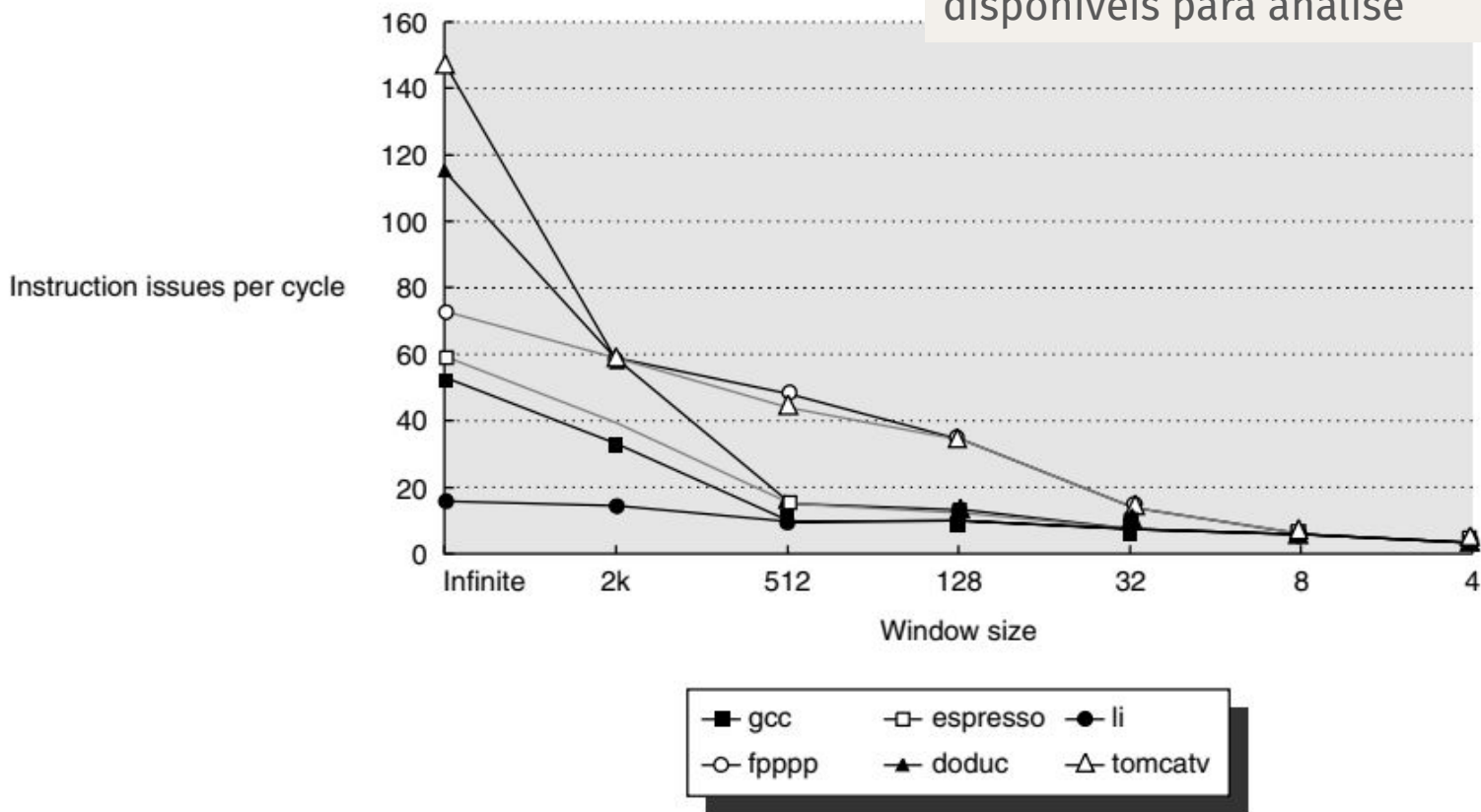
```
lw    t1, 500(a3)
```

# ILP disponível num processador perfeito

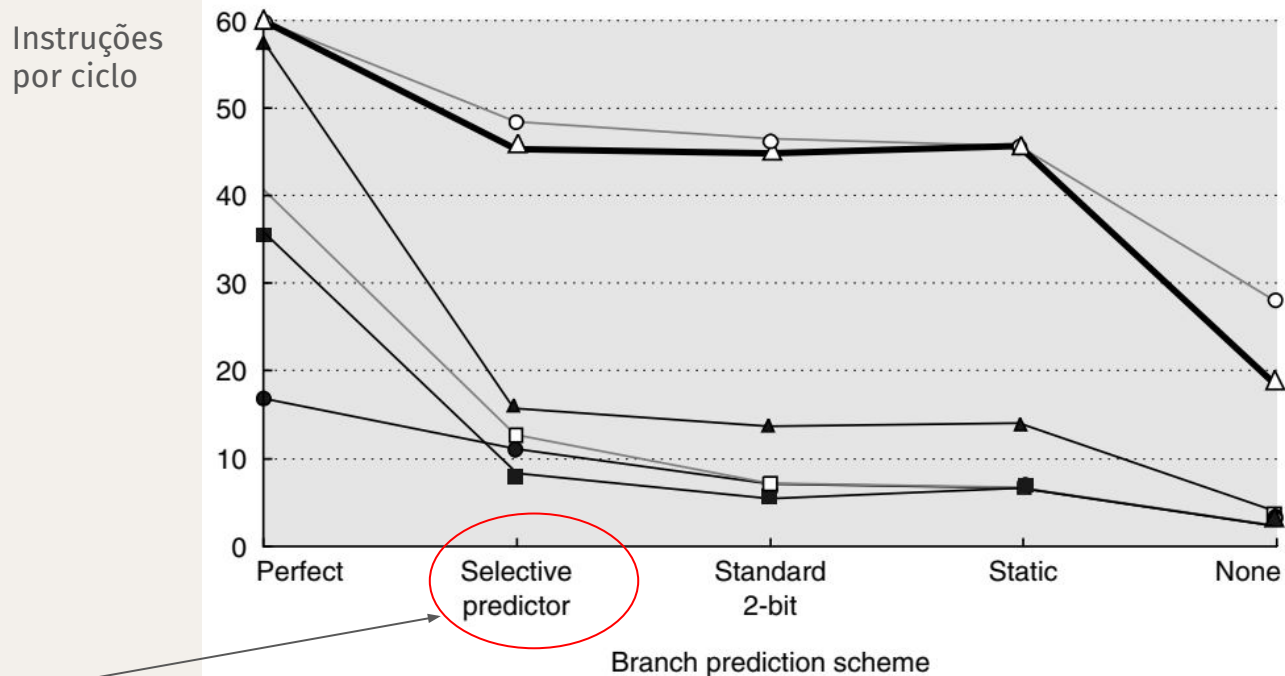


# Limitação: janela de instruções

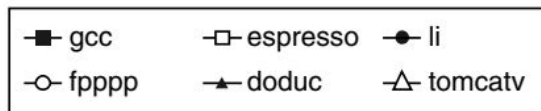
Janela: quantas instruções estão disponíveis para análise



# Limitação: qualidade da predição de saltos

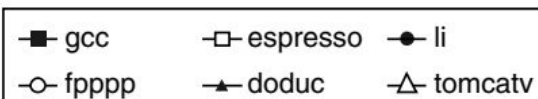
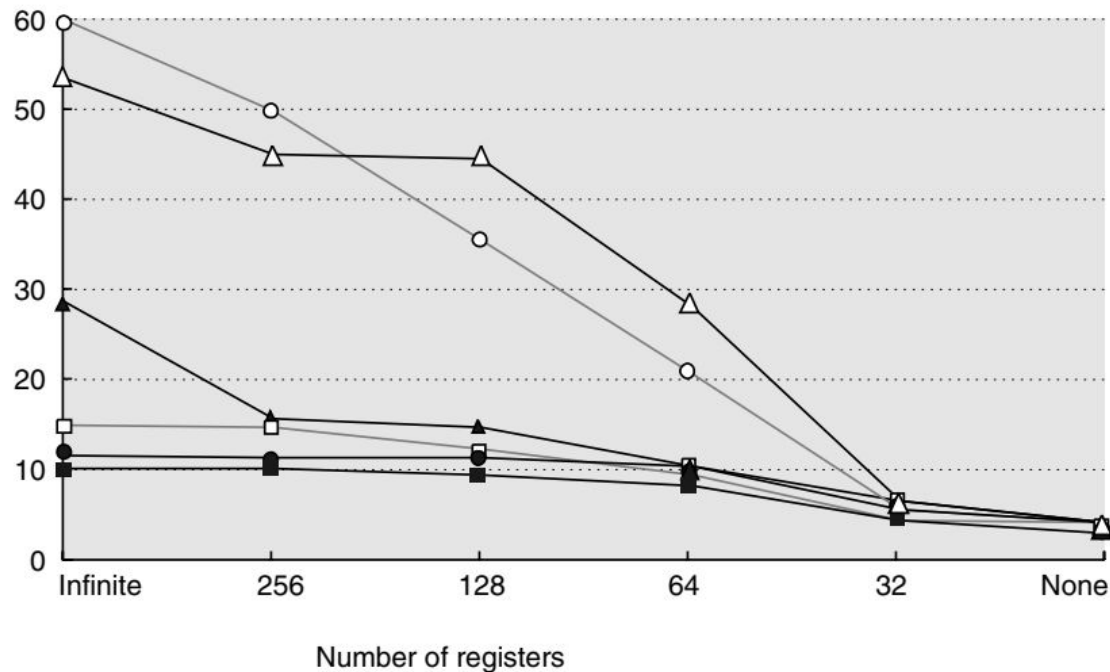


Não estudado



# Limitação: número de registos

Instruções  
por ciclo



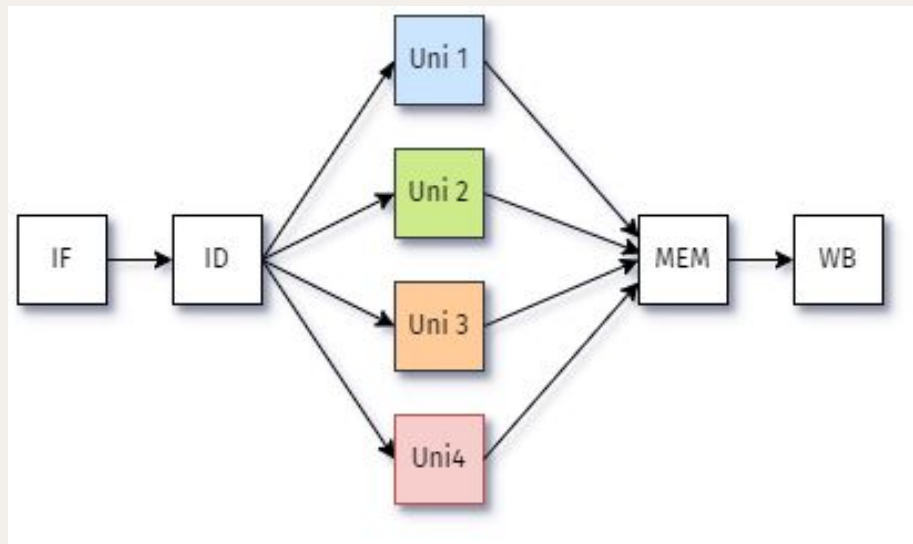


# Ordem de execução das instruções

# Execução de instruções por ordem do programa

- Instruções são iniciadas (emitidas) e executadas por ordem
  - Quando uma instrução faz parar a *pipeline* (*stall*), nenhuma instrução posterior pode ser executada.
- Trata-se de uma limitação importante quando **existem múltiplas unidades funcionais**.
- Atualmente, é fácil produzir circuitos com dezenas (ou mesmo centenas) de unidades funcionais.
- A existência de múltiplas unidades funcionais (por exemplo, ALUs) **reduz muito a existência de conflitos estruturais**.
  - Exemplo: num processador com 2 ALUs, é possível fazer duas adições simultaneamente.

# Várias unidades de cálculo: CPU multi-*pipeline*



Organização  
mais flexível!

- Unidades de cálculo podem ter características diferentes
  - Podem estar organizadas em *pipeline*
    - cada uma com número diferente de andares
  - Podem necessitar de vários ciclos para produzir o resultado
    - unidades multi-ciclo

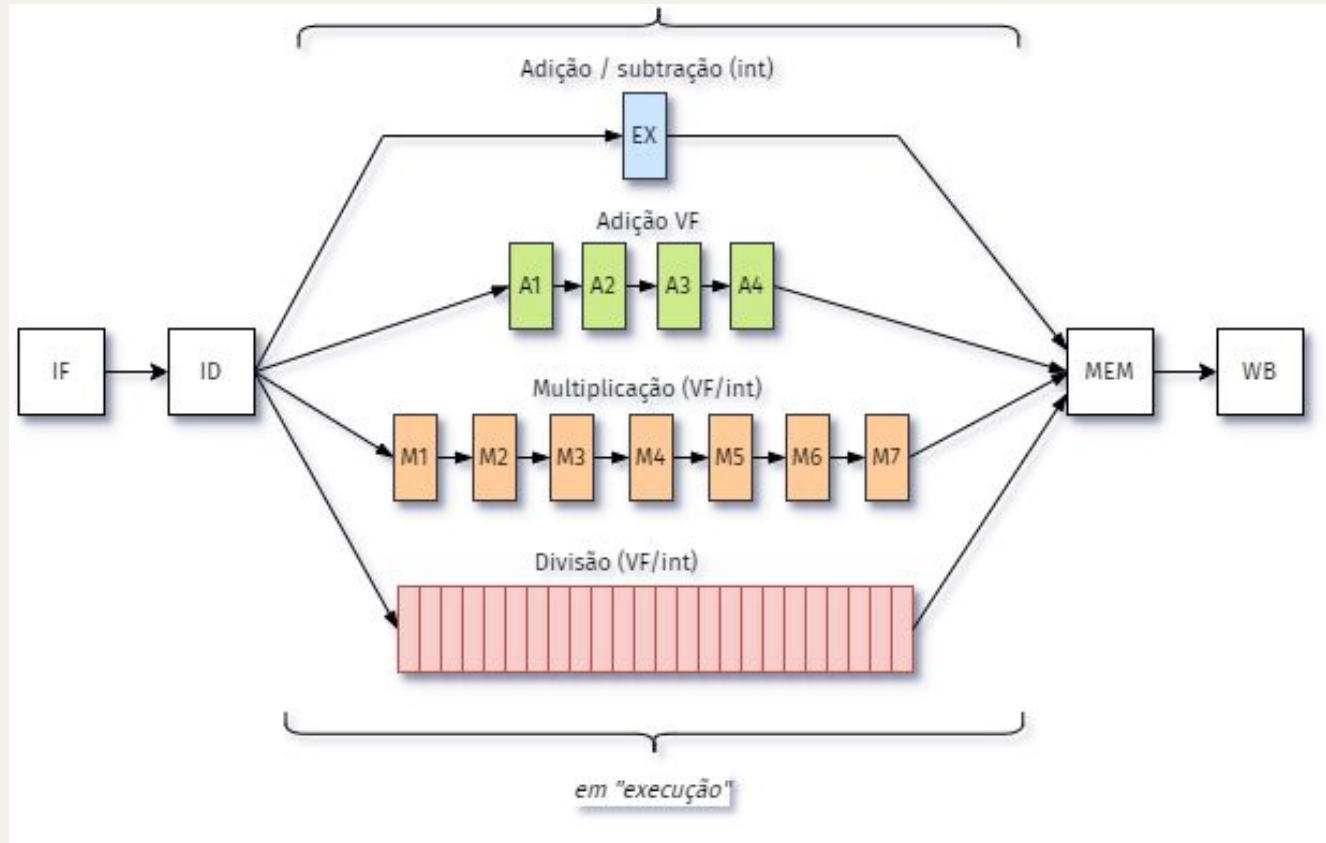
# Características das unidades funcionais

- Latência
  - Quantos ciclos **adicionais** são necessários para produzir o resultado?
    - Adicionais: para além do ciclo de relógio em que o processamento tem início
    - Mínimo: 0 (resultado está pronto no ciclo seguinte).
- Intervalo de iniciação (*initiation interval*)
  - Número mínimo de ciclos de relógio que devem decorrer entre o início de operações consecutivas na mesma unidade.
    - Valor mínimo: 1 ciclo (a unidade pode receber novos valores a cada ciclo de relógio)

Exemplo

| Unidade funcional             | Latência | II (Intervalo de iniciação) |
|-------------------------------|----------|-----------------------------|
| Adição e subtração (inteiros) | 0        | 1                           |
| Adição/subtração VF           | 3        | 1                           |
| Multiplicação (inteiros e VF) | 6        | 1                           |
| Divisão (inteiros e VF)       | 24       | 25                          |

# Organização de CPU multi-*pipeline* (referência)



# Regras básicas de funcionamento (multi-*pipeline*)

- Operações aritméticas suportadas
  - números inteiros: `div`, `mul`, `add`, `sub`, `lw`, `sw`; registos X0-X31
  - números em VF: `fdiv`, `fmul`, `fadd`, `fsub`, `flw`, `fsw`; registos F0-F31
- Tratamento de instruções em ID
  - Operandos da instrução são enviados para UF apropriada se:
    - A unidade funcional necessária estiver livre;
    - Não existirem dependências de dados entre a instrução em “emissão” e as instruções na fase de execução (i.e., cujos resultados não chegaram a MEM);
    - Estiver garantido que apenas uma instrução passa ao andar MEM em cada ciclo de relógio;
    - A ordem de passagem dos resultados ao andar MEM corresponder à ordem das operações no programa.
  - Quando estas condições não estão cumpridas, a emissão é protelada.
- Para simplificar, não se considera a existência de atalhos.

# Multi-*pipeline*: características mais importantes

- A emissão de instruções (passagem do andar ID para uma das unidades funcionais) é feita pela ordem do programa.
- Podem coexistir unidades funcionais com e sem organização em *pipeline*.
  - Na organização de referência, a unidade DIV necessita de 24 ciclos e não está organizada em *pipeline*.
- A parte final do tratamento de instruções (andares MEM e WB) é realizada pela ordem do programa.
- Várias unidades funcionais podem estar simultaneamente em operação (sujeito a restrições causadas pelas dependências).
- Esta organização de “*pipeline*” tem mais flexibilidade, mas ainda é limitada.
  - Cada unidade funcional pode ter nº diferente de andares de *pipeline*.

# Execução de instruções por ordem: exemplo

```
fdiv    f10, f12, f14  
fadd    f20, f10, f18  
fsub    f22, f18, f21
```

- A dependência de `fdiv` obriga a protelar a execução de `fadd`
  - É necessário esperar que o valor de `f10` seja calculado (24 ciclos!)
  - Atalhos não resolvem o problema.
- Contudo, a instrução `fsub` não tem dependências de dados em relação às outras instruções ⇒ **pode ser executada antes** da instrução `fadd`.
- As instruções são descodificadas (emitidas) em ordem mas seria vantajoso que pudessem passar para as unidades funcionais assim que os dados necessários estejam disponíveis.
  - **Execução fora de ordem** (*out-of-order execution*)
  - **Terminação fora de ordem** (*out-of-order completion*)



# Execução de instruções fora de ordem

- Emissão de instruções por ordem
  - Descodificação e verificação de conflitos estruturais
  - Aguardar (se necessário) a ausência de conflitos de dados
- Execução fora de ordem (*out-of-order execution*)
- Terminação fora de ordem (*out-of-order completion*)
- A execução de instruções fora de ordem pode causar **conflitos de outros tipos**.

Problema: Como escalonar (ordenar) a execução de instruções de forma a reduzir o número de ciclos de protelamento?

# Dependências: caracterização geral

1. Dependências de controlo
2. Dependências de dados (também designadas por “verdadeiras”)
  - Impõem uma ordem (parcial) de execução que não pode ser desrespeitada
  - O uso (leitura) de um valor deve obrigatoriamente seguir a sua produção (escrita)
3. Dependências de nome
  - Ocorrem quando duas instruções usam o mesmo registo ou posição de memória (têm o mesmo *nome*)
  - Mas **não existe fluxo de dados** entre as instruções associadas!
  - Apenas causam conflitos se as instruções forem executadas fora de ordem
  - Podem ser evitadas alterando o nome associado a uma das instruções
    - Relativamente fácil de implementar para registos.

# Dependência de nome: antidependência

- Antidependência entre instrução **x** e instrução **y**, se a instrução **y** escrever num registo (ou posição de memória) lido pela instrução **x**.

```
add    x1, x10, x2      # instrução x
sub    x10, x3, x4      # instrução y
```

- O exemplo mostra uma antidependência via registo x10.
- A ordem das instruções não pode ser trocada, embora não flua informação de **x** para **y**.
- Desrespeitar a dependência resulta num conflito WAR
  - WAR = *write after read*
  - O nome dos conflitos é derivado da condição que deve ser respeitada

# Dependência de nome: dependência de saída

- Existe uma dependência de saída entre a instrução **x** e a instrução **y**, se ambas as instruções escrevem no mesmo registro (ou posição de memória).

```
add    x1, x10, x2    # instrução x
sub    x1, x3, x4      # instrução y
```

- O exemplo mostra uma dependência de saída via registro x1.
- A ordem das instruções não pode ser trocada, embora não flua informação de **x** para **y**.
- Desrespeitar a dependência resulta num conflito WAW
  - WAW = *write after write*

# Dependências de dados (resumo)

| Conflito | Dependência     | Observação                                                                                                    |
|----------|-----------------|---------------------------------------------------------------------------------------------------------------|
| RAW      | verdadeira      | Instruções não podem ser reordenadas entre si.                                                                |
| WAR      | antidependência | Não ocorre na <i>pipeline</i> simples usada nos exemplos, mas pode ocorrer quando instruções são reordenadas. |
| WAW      | de saída        | Não ocorre na <i>pipeline</i> usada nos exemplos, mas pode ocorrer quando instruções são reordenadas.         |
| RAR      | nenhuma         | Não é um conflito; instruções podem ser reordenadas livremente.                                               |

# Escalonamento dinâmico

# Escalonamento dinâmico

- Estratégia para alterar a ordem da execução das instruções de forma a reduzir protelamentos e manter o fluxo de dados.
- Vantagens
  - O compilador não precisa de ter conhecimento da microarquitetura
  - Lida com casos em que as dependências são desconhecidas em tempo de compilação
  - Permite que o processador tolere atrasos imprevisíveis, como faltas da *cache*, executando outro código enquanto espera que a falta seja resolvida.
- Desvantagens
  - Leva a aumento substancial da complexidade do *hardware*.

# Escalonamento dinâmico melhora desempenho

- Escalonamento dinâmico implica
  - Execução fora de ordem
  - Terminação (da execução) fora de ordem

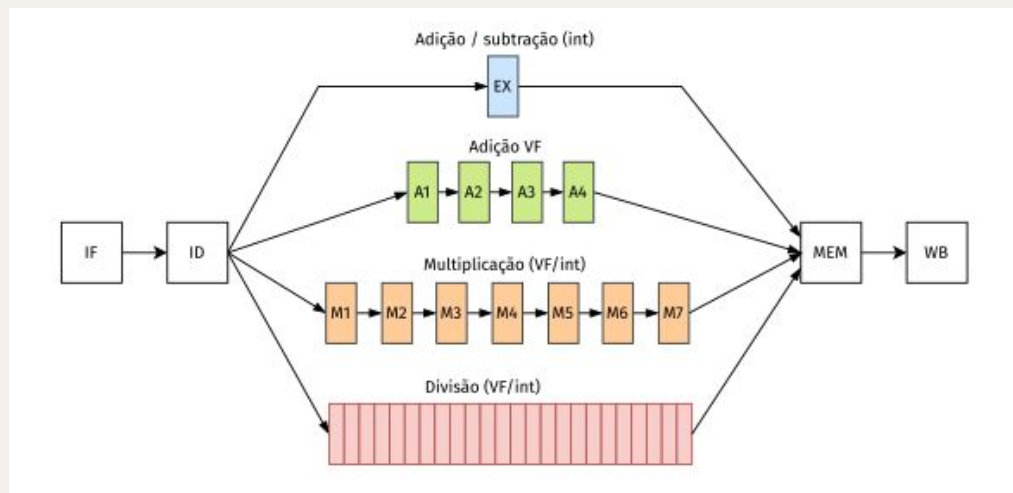
## Exemplo do problema

fdiv **f10**, f2, f4

fadd f10, **f10**, f8

**fsub** f12, f8, f14

Com escalonamento dinâmico, **fsub** pode ser executada antes de **fadd**, porque não está impedida por dependências de dados.





# Execução fora de ordem pode produzir outros conflitos

- Execução fora de ordem pode levar a conflitos
  - WAR (desrespeitar uma antidependência)
  - WAW (desrespeitar uma dependência de saída)

## Exemplo do problema

fdiv     **f10**, f2, f4  
fmul     f6, **f10**, f8  
fadd     **f10**, f11, f14

antidependência

dependência de saída

Executar **fadd** antes de **fmul** causa conflito  
WAR (**fmul** usa valor errado de **f10**)

Atualizar **f10** antes de **fdiv** causa  
conflito WAW

# Solução: introduzir registos adicionais

- Usar registos “extra” para remover dependências de nome
  - “Register renaming”

```
fdiv      f0, f2, f4
fadd      f6, f0, f8
fsw       f6, 0(t1)
fsub      f8, f10, f14
fmul      f6, f10, f8
```



```
fdiv      f0, f2, f4
fadd      S, f0, f8
fsd       S, 0(t1)
fsub      T, f10, f14
fmul      f6, f10, T
```

- *anti-dependências*
  - fadd / fsub (via f8)
  - fsw / fmul (via f6)
- *dependência de saída*
  - fadd / fmul (via f6)

- dois registos temporários S e T
- usos subsequentes de f8 substituídos por T
- ainda ficam dependências de dados “verdadeiras”

# Abordagem geral

Para permitir uma execução fora de ordem, dividir ID em duas fases:

1. Emissão

Descodificação de instruções, verificação de conflitos estruturais.

2. Leitura de operandos

Esperar até que não haja conflitos de dados, depois ler os operandos.

- O módulo de busca de instruções (IF)
  - precede o módulo de emissão
  - pode colocar as instruções num registo de instruções ou numa fila de instruções pendentes;
  - as instruções são então emitidas a partir do registo ou da fila.
- A fase de execução (EX) segue-se à fase de leitura dos operandos
  - A execução pode demorar vários ciclos, dependendo da operação.

# Estado de uma instrução

- Distinguimos quando uma instrução começa a ser executada e quando termina a execução:
  - entre os dois momentos, a instrução está em execução.
- *Pipeline* deve permitir ter várias instruções em execução ao mesmo tempo.
- Ter múltiplas instruções em execução ao mesmo tempo requer
  - múltiplas unidades funcionais (com vários ciclos, dependendo da unidade)
  - ou unidades funcionais em *pipeline*,
  - ou ambas.
- Como as duas capacidades são essencialmente equivalentes para efeitos de controlo de *pipeline*, assumiremos que o processador tem múltiplas unidades funcionais.

# Escalonamento dinâmico e renomeação de registos

- Todas as instruções passam pela fase de emissão por ordem.
- Podem contudo ser bloqueadas (à espera de operandos) na segunda fase (leitura de operandos), entrando assim na fase de execução fora de ordem.
- O **algoritmo de Tomasulo** permite gerir dinamicamente o escalonamento da execução das instruções.
- Também lida com antidependências e dependências de saída através da renomeação dos registos.
- O algoritmo de Tomasulo pode ser estendido para suportar especulação:
  - técnica para reduzir o efeito das dependências de controlo
    - prevendo o resultado de um salto condicional;
    - prosseguindo com a execução das instruções no endereço de destino previsto;
    - tomando medidas corretivas quando a previsão está errada.

# Implementação: método de Tomasulo

- Método de Tomasulo:
  - Determina disponibilidade dos operandos
    - Evita conflitos RAW
  - Introduce a renomeação de registos (*register renaming*) em *hardware*
    - Evita os conflitos WAW e WAR
- A renomeação de registos é feita por **estações de reserva** (ER) contendo:
  - A operação
  - Valores dos operandos armazenados (quando disponíveis)
  - Número da estação de reserva da instrução que fornece os valores dos operandos
- Uma estação de reserva armazena um operando assim que ele está disponível, eliminando a necessidade de obter o operando de um registo.
- Um resultado é distribuído a todas as estações de reserva via um barramento comum (CDB = *common data bus*)

# Estações de reserva e barramento comum

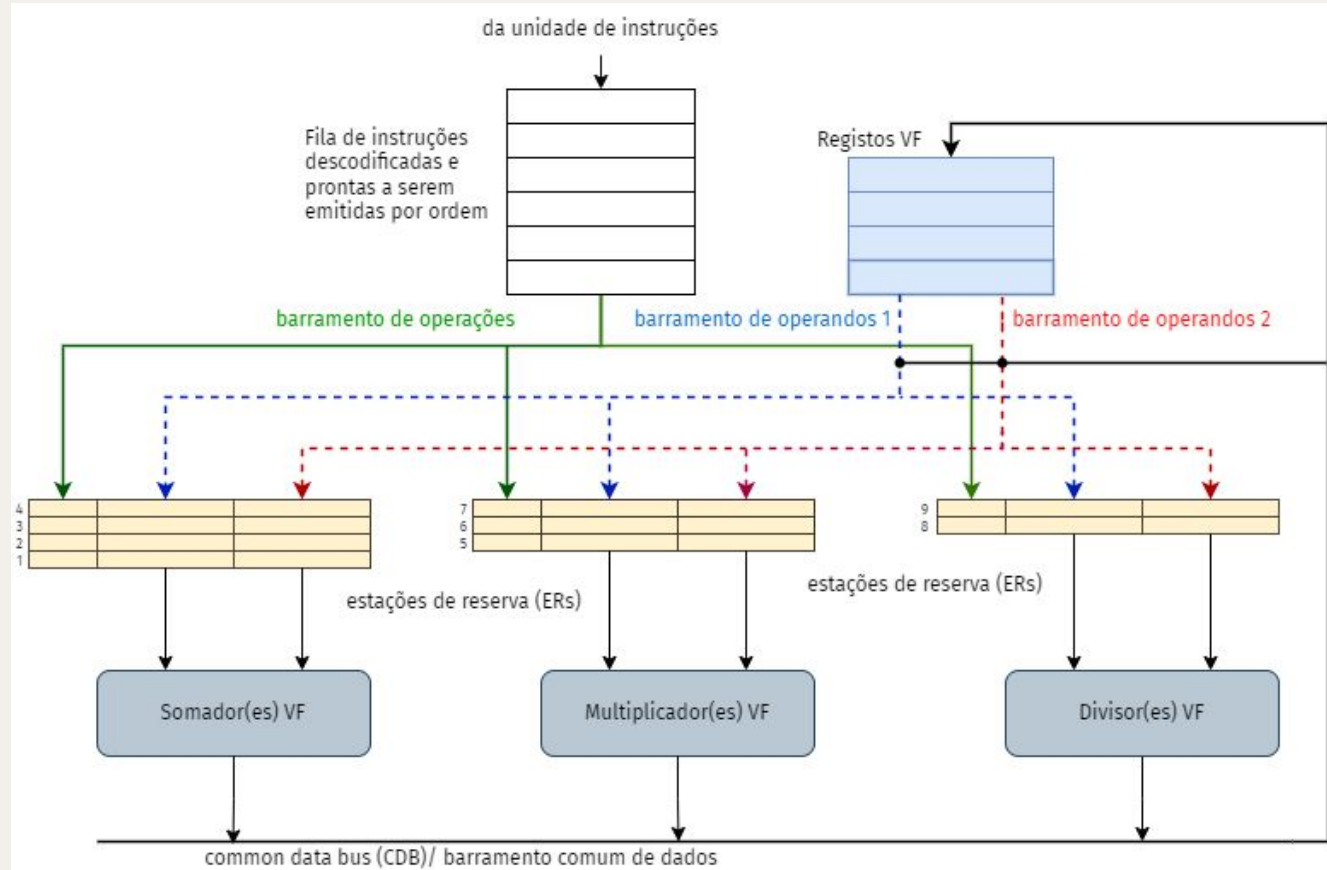
- O método de Tomasulo usa estações de reserva (ER) para além do banco de registos centralizado.
  - Conjunto de registos associado a uma ou mais UFs iguais
- A deteção de conflitos e o controlo da execução são distribuídos:
  - A informação contida nas estações de reserva de cada UF determina quando uma instrução pode começar a ser executada.
- A transferência de resultados é feita por um barramento de resultados comum que permite que todas as unidades que aguardam um operando sejam atualizadas simultaneamente.

# Regras de funcionamento (escalonamento dinâmico)

- Cada registo de uma ER armazena operandos assim que estes ficam disponíveis (dos registos ou do barramento de dados comum).
  - Instruções residentes numa ER registam qual a ER para a qual enviarão o seu resultado.
  - Se os valores do operando para uma instrução tiverem sido calculados, também são armazenados no registo; caso contrário, o registo da ER guarda os nomes (números) das estações de reserva que fornecerão os valores do operando.
- (Renomeação de registos)**
- Podem existir mais estações de reserva do que registos.



# Organização para método de Tomasulo (exemplo)



# Algoritmo de Tomasulo: fases

- Emitir (ou despachar)
  - Obter a próxima instrução da fila FIFO (*first in, first out*)
  - Se houver uma ER disponível, emitir a instrução para a ER com os valores dos operandos, se disponíveis.
    - Se os valores dos operandos não estiverem disponíveis, indicar as ERs que os produzirão. (evita conflitos WAW e WAR)
  - Se não existirem estações livres, protelar.
- Executar
  - Quando os operandos estiverem prontos e unidade funcional estiver livre, dar início à execução do cálculo. (evita conflitos RAW)
- Escrever resultado
  - Guardar o resultado via CDB nas estações de reserva e no banco de registos
  - Assumir que, quando existem vários valores disponíveis no mesmo ciclo de relógio, o resultado da operação mais antiga é transferido primeiro.

# Registos adicionais

- Na implementação de referência, os registos extra (não mencionados nas instruções) usados para renomeação são 18 (2 para cada posição):
  - 8 na ER associada aos somadores, 6 na dos multiplicadores e 4 na dos divisores
- A ER pode ser representada (no exemplo) por um número de 4 bits.
- Num processador com mais registos “visíveis” (especificados na ISA), o número de registos “extra” também seria maior.
  - No exemplo, existem apenas 4 registos de vírgula flutuante.
- As etiquetas (nomes de registos privados) no esquema Tomasulo referem-se à estação de reserva que irá produzir um resultado.
  - Os nomes dos registos “visíveis” são conceptualmente descartados quando uma instrução é emitida para uma estação de reserva.

# Sumário da informação interna necessária

Cada ER tem 6 campos:

- Op — A operação a executar.
- Qj, Qk — As estações de reserva que produzirão os operandos (zero indica que o operando já está disponível em Vj ou Vk)
- Vj, Vk — O valor dos operandos.
- Busy — Indica que a estação de reserva está ocupada.

O banco de registos mantém informação associada a um registo “visível”

- Qi — O número da estação de reserva que contém a operação cujo resultado deve ser armazenado neste registo; 0 se não houver nenhuma.

# Exemplo de execução

Mostrar o conteúdo das tabelas de informação para a sequência de código indicada a seguir:

|    |      |            |
|----|------|------------|
| 1. | fmul | f1, f3, f4 |
| 2. | fsub | f3, f2, f4 |
| 3. | fdiv | f5, f5, f3 |
| 4. | fadd | f2, f4, f6 |

## Recursos do CPU

- Seis registos (f1 - f6)
- ERs de unidade de adição/subtração: 1, 2, 3
- ERs de unidade de multiplicação: 4, 5
- ER da unidade de divisão: 6

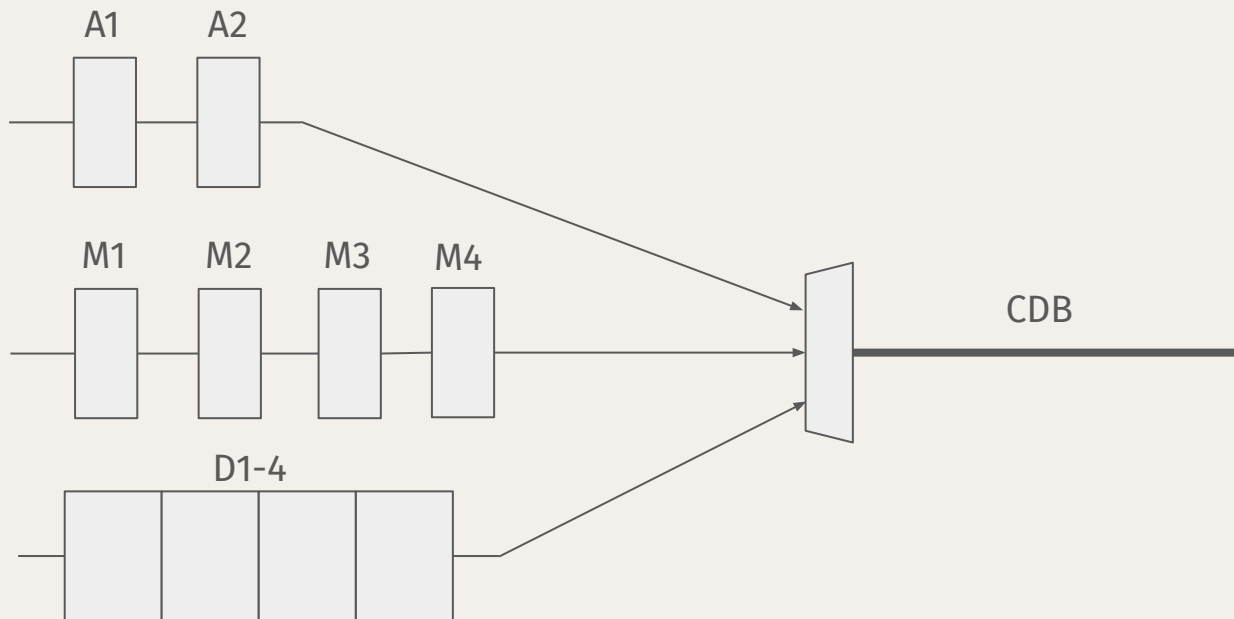
*Assumir os seguintes valores para a latência das unidades funcionais necessárias:*

*Adição ou subtração: 2 ciclos; multiplicação: 4 ciclos; divisão: 4 ciclos.*

*O II é de 1 ciclo para todas as unidades, exceto para a unidade de divisão, que tem II=5.*

# Registos do CPU e unidades funcionais (situação inicial)

| REG   | F1  | F2  | F3  | F4  | F5  | F6  |
|-------|-----|-----|-----|-----|-----|-----|
| valor | 1.5 | 2.3 | 3.7 | 4.9 | 5.1 | 6.4 |



# Exemplo: Ciclo 1

- Instrução 1 vai para uma das estações de reserva da unidade de multiplicação.
- Os operandos são obtidos do banco de registos.



**fmul f1, f3, f4**

| ER | Busy | OP | Qj | Qk | Vj | Vk |
|----|------|----|----|----|----|----|
| 1  | 0    |    |    |    |    |    |
| 2  | 0    |    |    |    |    |    |
| 3  | 0    |    |    |    |    |    |
| 4  | 0    |    |    |    |    |    |
| 5  | 0    |    |    |    |    |    |
| 6  | 0    |    |    |    |    |    |

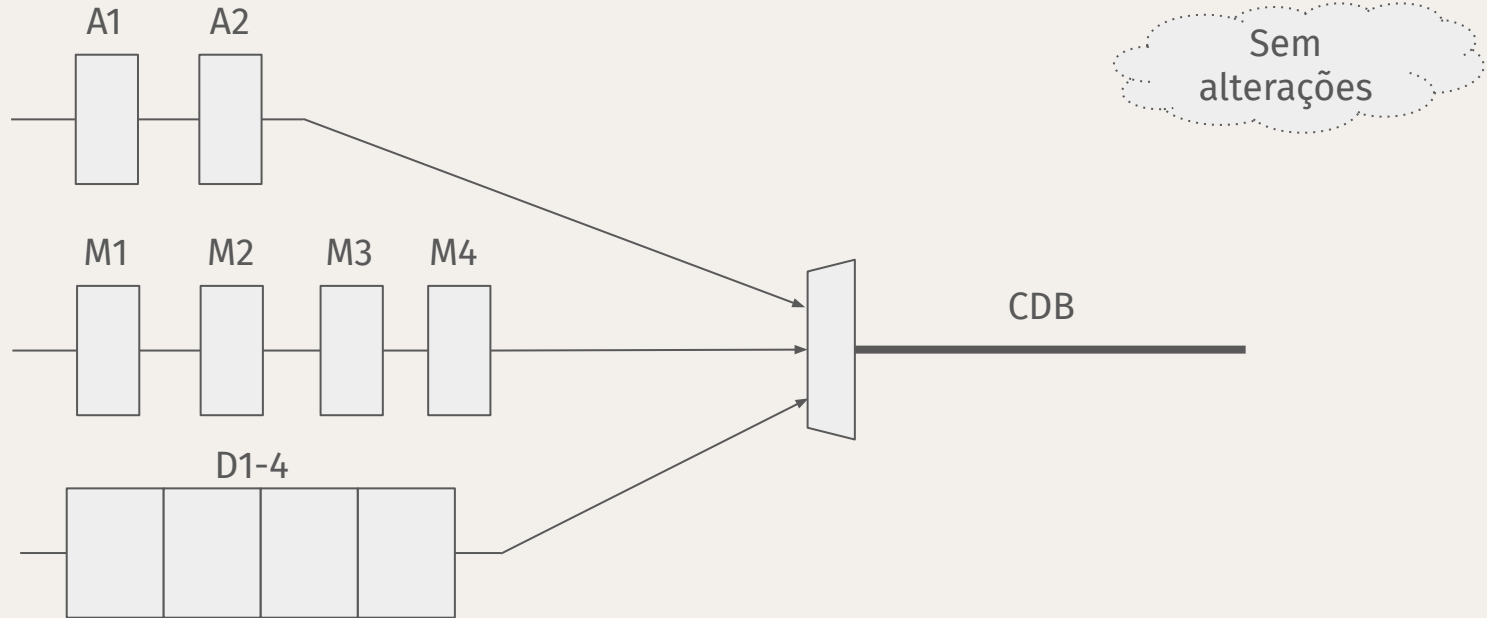
| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 0  | 0  | 0  | 0  | 0  | 0  |

| ER | Busy | OP   | Qj | Qk | Vj  | Vk  |
|----|------|------|----|----|-----|-----|
| 1  | 0    |      |    |    |     |     |
| 2  | 0    |      |    |    |     |     |
| 3  | 0    |      |    |    |     |     |
| 4  | 1    | fmul | 0  | 0  | 3.7 | 4.9 |
| 5  | 0    |      |    |    |     |     |
| 6  | 0    |      |    |    |     |     |

| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 4  | 0  | 0  | 0  | 0  | 0  |

# Registos do CPU e unidades funcionais (fim do ciclo 1)

| REG   | F1  | F2  | F3  | F4  | F5  | F6  |
|-------|-----|-----|-----|-----|-----|-----|
| valor | 1.5 | 2.3 | 3.7 | 4.9 | 5.1 | 6.4 |





## Exemplo (continuação): ciclo 2

- Instrução 2 (fsub) pode ser enviada para uma ER da unidade de adição/subtração.
- Instrução na ER 4 (fmul) pode passar para UF (disponível a partir do ciclo 6).



fsub    f3, f2, f4

| ER | Busy | OP   | Qj | Qk | Vj  | Vk  |
|----|------|------|----|----|-----|-----|
| 1  | 0    |      |    |    |     |     |
| 2  | 0    |      |    |    |     |     |
| 3  | 0    |      |    |    |     |     |
| 4  | 1    | fmul | 0  | 0  | 3.7 | 4.9 |
| 5  | 0    |      |    |    |     |     |
| 6  | 0    |      |    |    |     |     |

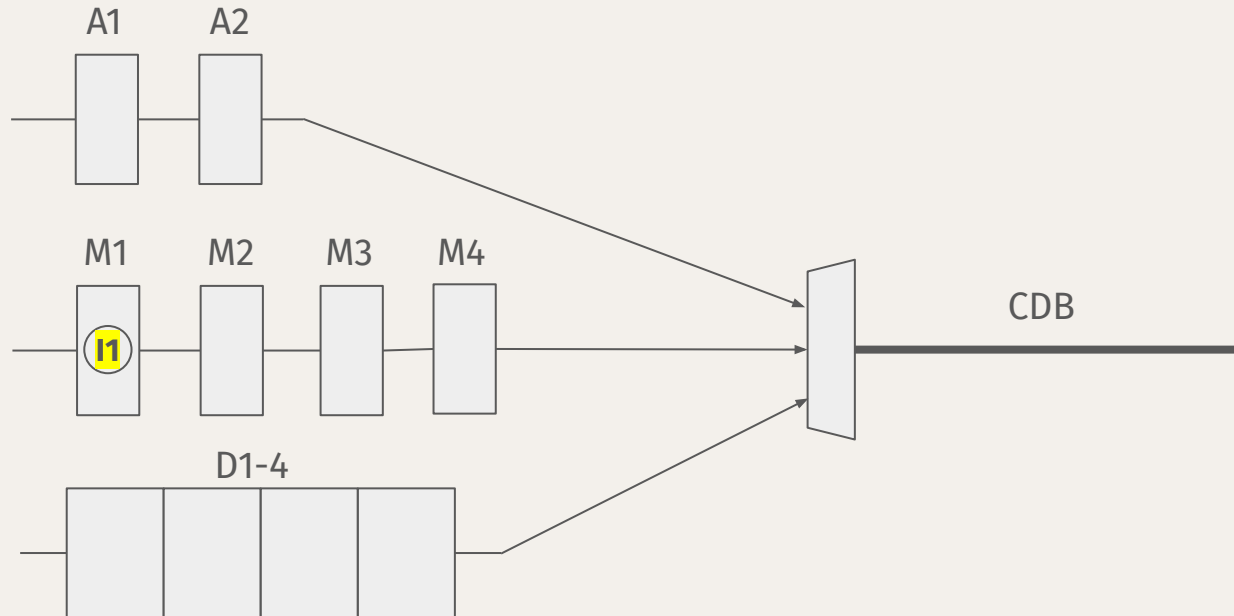
| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 4  | 0  | 0  | 0  | 0  | 0  |

| ER | Busy | OP   | Qj | Qk | Vj  | Vk  |
|----|------|------|----|----|-----|-----|
| 1  | 1    | fsub | 0  | 0  | 2.3 | 4.9 |
| 2  | 0    |      |    |    |     |     |
| 3  | 0    |      |    |    |     |     |
| 4  | 1    | fmul | 0  | 0  | 3.7 | 4.9 |
| 5  | 0    |      |    |    |     |     |
| 6  | 0    |      |    |    |     |     |

| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 4  | 0  | 1  | 0  | 0  | 0  |

# Registos do CPU e unidades funcionais (fim do ciclo 2)

| REG   | F1  | F2  | F3  | F4  | F5  | F6  |
|-------|-----|-----|-----|-----|-----|-----|
| valor | 1.5 | 2.3 | 3.7 | 4.9 | 5.1 | 6.4 |



## Exemplo (continuação): ciclo 3

- Instrução 3 (fdiv) pode ser enviada para a ER da unidade de divisão (existe uma ER livre)
- Instrução 2 (na ER 1) pode passar para UF (nota: resultado no ciclo 4, usável no ciclo 5)



fdiv f5, f5, f3

| ER | Busy | OP   | Qj | Qk | Vj  | Vk  |
|----|------|------|----|----|-----|-----|
| 1  | 1    | fsub | 0  | 0  | 2.3 | 4.9 |
| 2  | 0    |      |    |    |     |     |
| 3  | 0    |      |    |    |     |     |
| 4  | 1    | fmul | 0  | 0  | 3.7 | 4.9 |
| 5  | 0    |      |    |    |     |     |
| 6  | 0    |      |    |    |     |     |

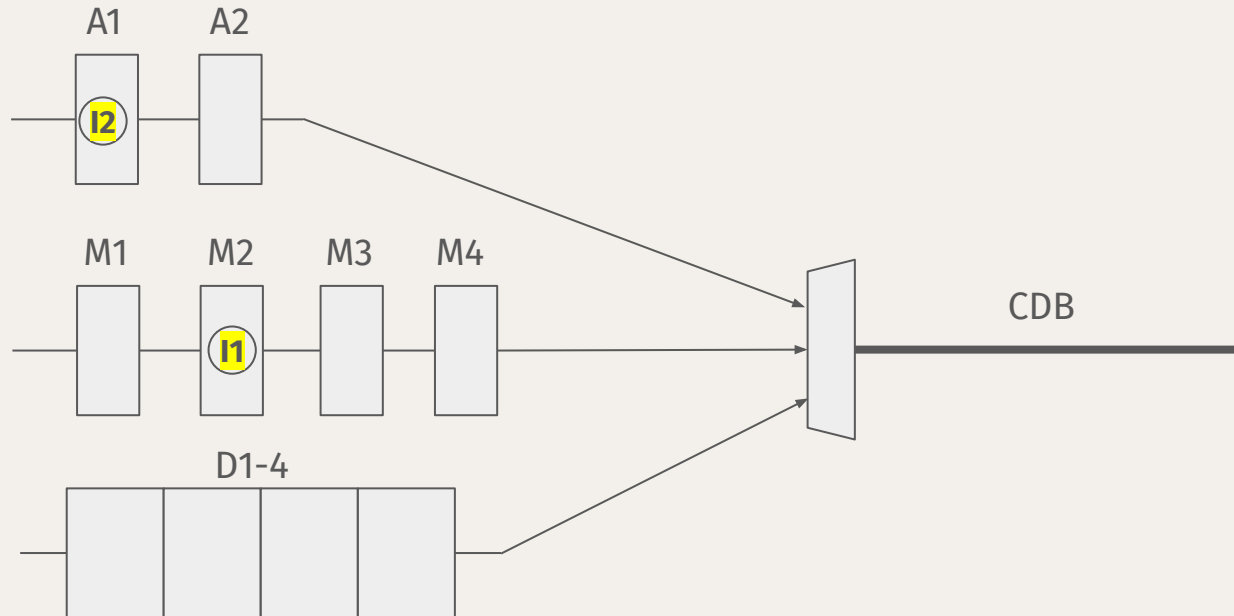
| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 4  | 0  | 1  | 0  | 0  | 0  |

| ER | Busy | OP   | Qj | Qk | Vj  | Vk  |
|----|------|------|----|----|-----|-----|
| 1  | 1    | fsub | 0  | 0  | 2.3 | 4.9 |
| 2  | 0    |      |    |    |     |     |
| 3  | 0    |      |    |    |     |     |
| 4  | 1    | fmul | 0  | 0  | 3.7 | 4.9 |
| 5  | 0    |      |    |    |     |     |
| 6  | 1    | fdiv | 0  | 1  | 5.1 | ?   |

| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 4  | 0  | 1  | 0  | 6  | 0  |

# Registos do CPU e unidades funcionais (fim do ciclo 3)

| REG   | F1  | F2  | F3  | F4  | F5  | F6  |
|-------|-----|-----|-----|-----|-----|-----|
| valor | 1.5 | 2.3 | 3.7 | 4.9 | 5.1 | 6.4 |



## Exemplo (continuação): ciclo 4

- Instrução 4 (fadd) pode ser enviada para a ER da unidade de adição/subtração (estão 2 disponíveis).
- Instrução na ER 6 (fdiv) não pode sair da ER (falta operando).



fadd f2, f4, f6

| ER | Busy | OP   | Qj | Qk | Vj  | Vk  |
|----|------|------|----|----|-----|-----|
| 1  | 1    | fsub | 0  | 0  | 2.3 | 4.9 |
| 2  | 0    |      |    |    |     |     |
| 3  | 0    |      |    |    |     |     |
| 4  | 1    | fmul | 0  | 0  | 3.7 | 4.9 |
| 5  | 0    |      |    |    |     |     |
| 6  | 1    | fdiv | 0  | 1  | 5.1 | ?   |

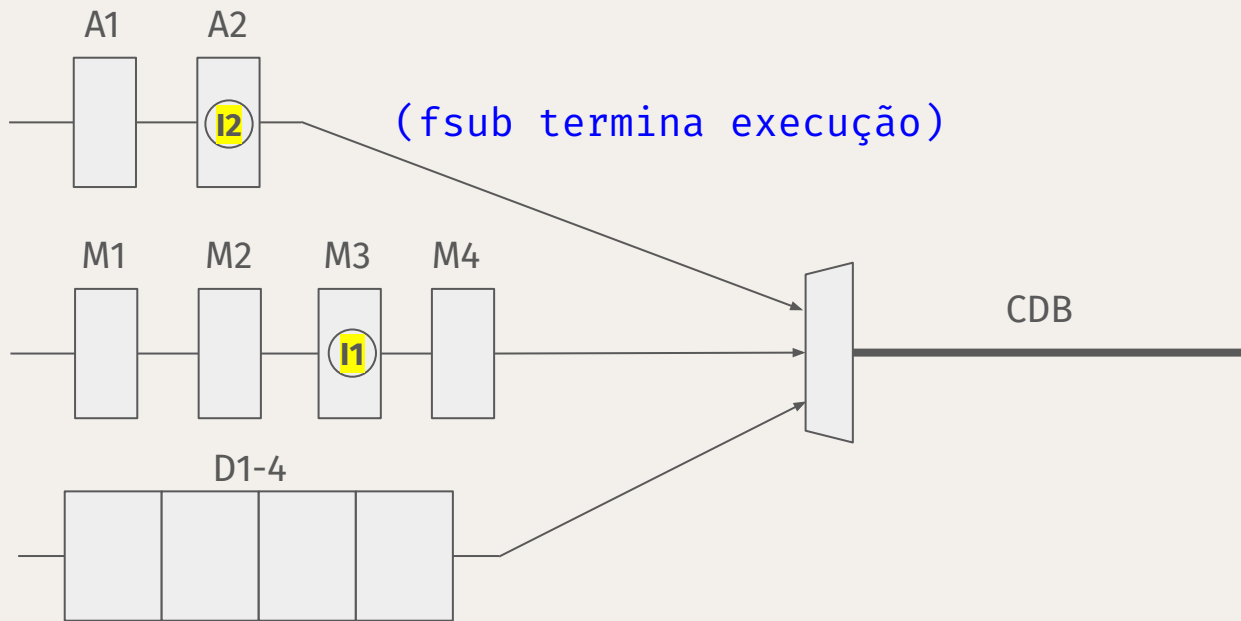
| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 4  | 0  | 1  | 0  | 6  | 0  |

| ER | Busy | OP   | Qj | Qk | Vj  | Vk  |
|----|------|------|----|----|-----|-----|
| 1  | 1    | fsub | 0  | 0  | 2.3 | 4.9 |
| 2  | 1    | fadd | 0  | 0  | 4.9 | 6.4 |
| 3  | 0    |      |    |    |     |     |
| 4  | 1    | fmul | 0  | 0  | 3.7 | 4.9 |
| 5  | 0    |      |    |    |     |     |
| 6  | 1    | fdiv | 0  | 1  | 5.1 | ?   |

| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 4  | 2  | 1  | 0  | 6  | 0  |

# Registos do CPU e unidades funcionais (fim do ciclo 4)

| REG   | F1  | F2  | F3  | F4  | F5  | F6  |
|-------|-----|-----|-----|-----|-----|-----|
| valor | 1.5 | 2.3 | 3.7 | 4.9 | 5.1 | 6.4 |



## Exemplo (continuação): ciclo 5

- ER1: Valor de registo interno A2 pode passar para CDB (produz resultado de ER1) e atualiza F3.
- Instrução na ER6 (fdiv) tem agora o valor de todos os operandos (usa valor de ER1).



| ER | Busy | OP   | Qj | Qk | Vj  | Vk  |
|----|------|------|----|----|-----|-----|
| 1  | 1    | fsub | 0  | 0  | 2.3 | 4.9 |
| 2  | 1    | fadd | 0  | 0  | 4.9 | 6.4 |
| 3  | 0    |      |    |    |     |     |
| 4  | 1    | fmul | 0  | 0  | 3.7 | 4.9 |
| 5  | 0    |      |    |    |     |     |
| 6  | 1    | fdiv | 0  | 1  | 5.1 | ?   |

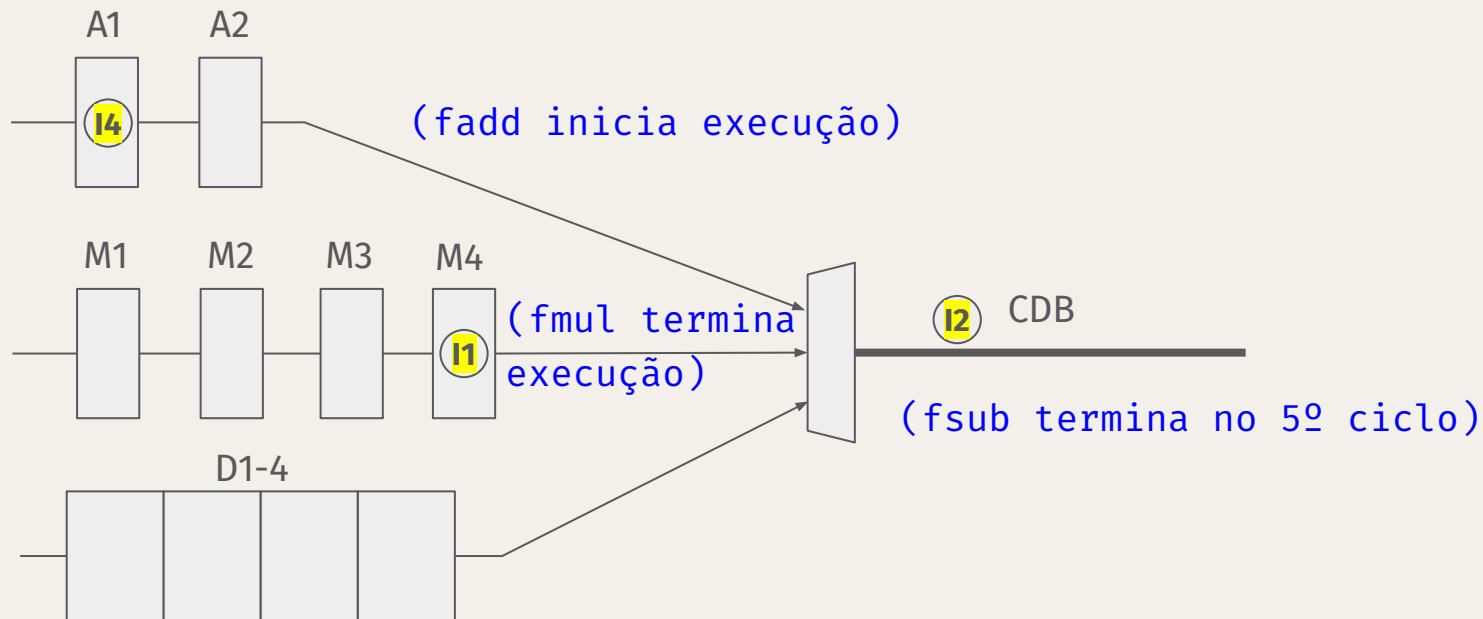
| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 4  | 2  | 1  | 0  | 6  | 0  |

| ER | Busy | OP   | Qj | Qk | Vj  | Vk   |
|----|------|------|----|----|-----|------|
| 1  | 0    |      |    |    |     |      |
| 2  | 1    | fadd | 0  | 0  | 4.9 | 6.4  |
| 3  | 0    |      |    |    |     |      |
| 4  | 1    | fmul | 0  | 0  | 3.7 | 4.9  |
| 5  | 0    |      |    |    |     |      |
| 6  | 1    | fdiv | 0  | 0  | 5.1 | -2.6 |

| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 4  | 2  | 0  | 0  | 6  | 0  |

# Registos do CPU e unidades funcionais (fim do ciclo 5)

| REG   | F1  | F2  | F3   | F4  | F5  | F6  |
|-------|-----|-----|------|-----|-----|-----|
| valor | 1.5 | 2.3 | -2.6 | 4.9 | 5.1 | 6.4 |





## Exemplo (continuação): ciclo 6

- Instrução 3 (fdiv) pode ser enviada para a ER de divisão (que está livre)
- Resultado da instrução 3 (ER4) é disponibilizado no CDB (atualiza F1)
- Instrução na ER 2 pode passar para UF (resultado no ciclo 8)



| ER | Busy | OP   | Qj | Qk | Vj  | Vk   |
|----|------|------|----|----|-----|------|
| 1  | 0    |      |    |    |     |      |
| 2  | 0    | fadd | 0  | 0  | 4.9 | 6.4  |
| 3  | 0    |      |    |    |     |      |
| 4  | 1    | fmul | 0  | 0  | 3.7 | 4.9  |
| 5  | 0    |      |    |    |     |      |
| 6  | 1    | fdiv | 0  | 0  | 5.1 | -2.6 |

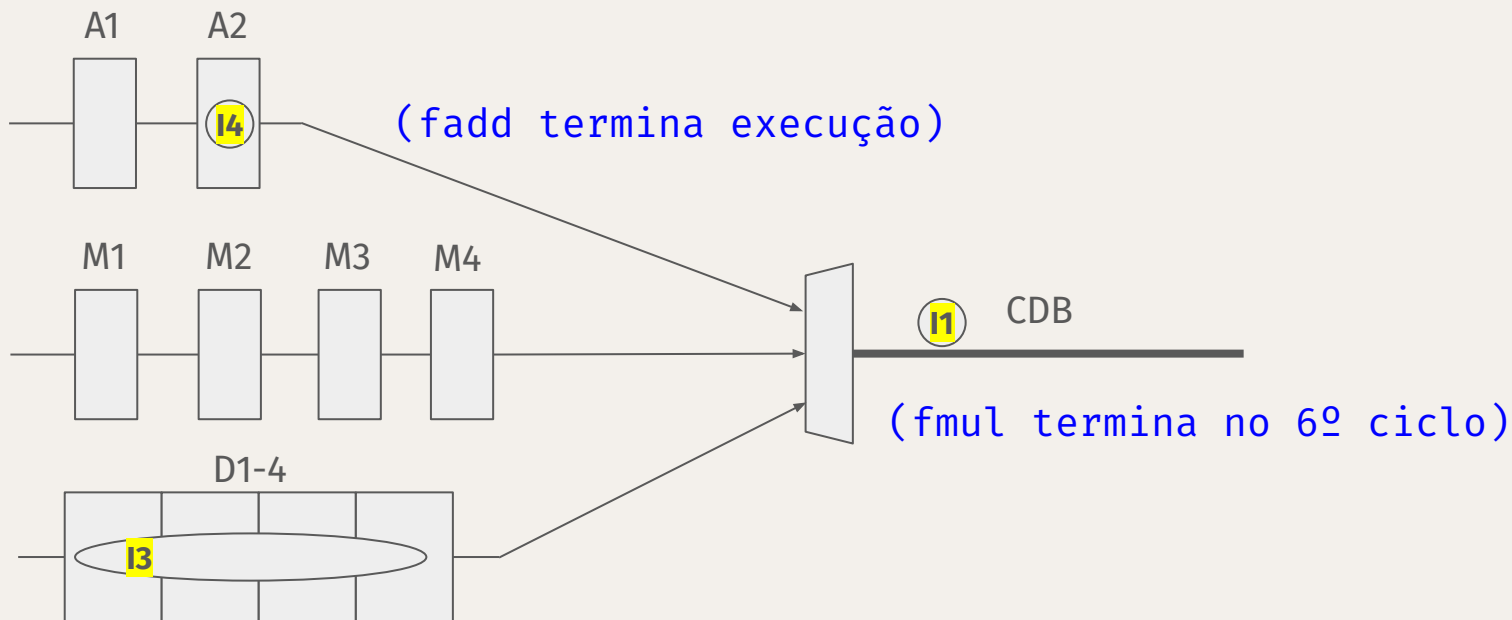
| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 4  | 2  | 0  | 0  | 6  | 0  |

| ER | Busy | OP   | Qj | Qk | Vj  | Vk   |
|----|------|------|----|----|-----|------|
| 1  | 0    |      |    |    |     |      |
| 2  | 1    | fadd | 0  | 0  | 4.9 | 6.4  |
| 3  | 0    |      |    |    |     |      |
| 4  | 0    |      |    |    |     |      |
| 5  | 0    |      |    |    |     |      |
| 6  | 1    | fdiv | 0  | 0  | 5.1 | -2.6 |

| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 0  | 2  | 0  | 0  | 6  | 0  |

# Registos do CPU e unidades funcionais (fim do ciclo 6)

| REG   | F1    | F2  | F3   | F4  | F5  | F6  |
|-------|-------|-----|------|-----|-----|-----|
| valor | 18.13 | 2.3 | -2.6 | 4.9 | 5.1 | 6.4 |



# Exemplo (continuação): ciclo 7

- Resultado da instrução 4 (ER2) é disponibilizado no CDB (atualiza F2)

| ER | Busy | OP   | Qj | Qk | Vj  | Vk   |
|----|------|------|----|----|-----|------|
| 1  | 0    |      |    |    |     |      |
| 2  | 0    | fadd | 0  | 0  | 4.9 | 6.4  |
| 3  | 0    |      |    |    |     |      |
| 4  | 0    |      |    |    |     |      |
| 5  | 0    |      |    |    |     |      |
| 6  | 1    | fdiv | 0  | 0  | 5.1 | -2.6 |

| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 0  | 2  | 0  | 0  | 6  | 0  |

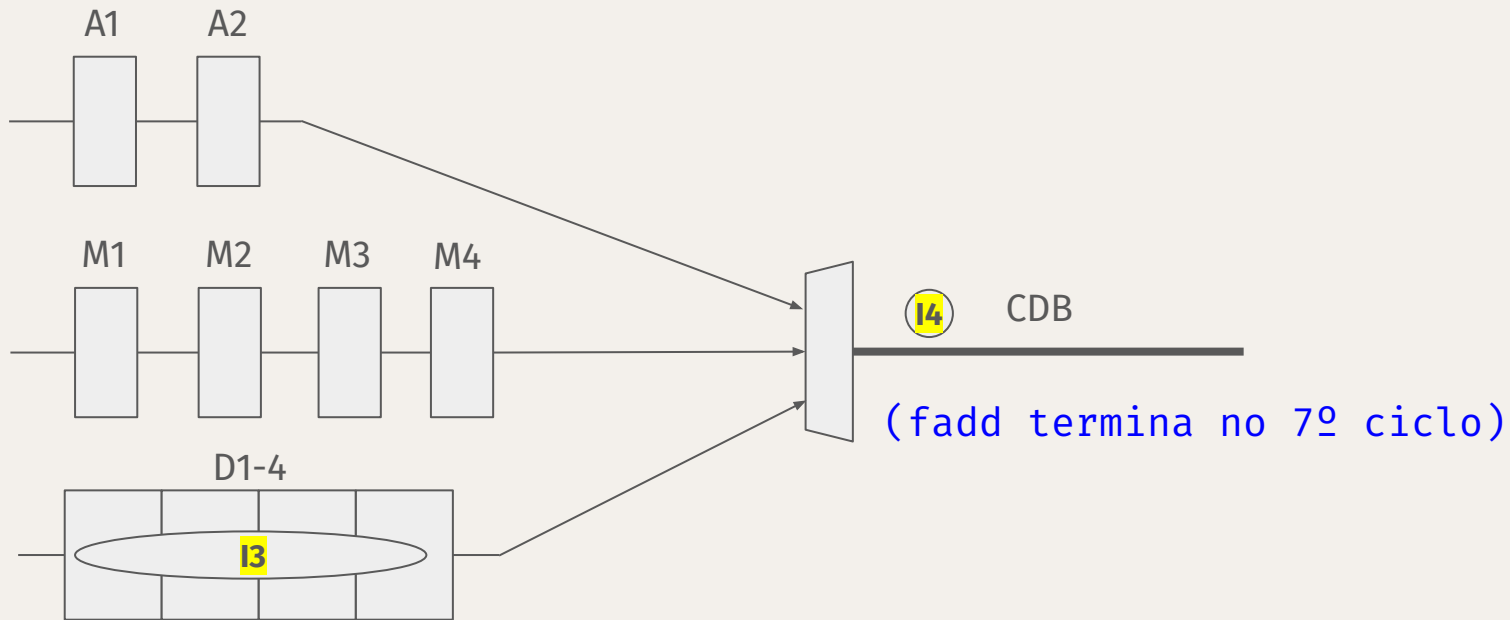


| ER | Busy | OP   | Qj | Qk | Vj  | Vk   |
|----|------|------|----|----|-----|------|
| 1  | 0    |      |    |    |     |      |
| 2  | 0    |      |    |    |     |      |
| 3  | 0    |      |    |    |     |      |
| 4  | 0    |      |    |    |     |      |
| 5  | 0    |      |    |    |     |      |
| 6  | 1    | fdiv | 0  | 0  | 5.1 | -2.6 |

| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 0  | 0  | 0  | 0  | 6  | 0  |

# Registos do CPU e unidades funcionais (fim do ciclo 7)

| REG   | F1    | F2   | F3   | F4  | F5  | F6  |
|-------|-------|------|------|-----|-----|-----|
| valor | 18.13 | 11.3 | -2.6 | 4.9 | 5.1 | 6.4 |



# Exemplo (continuação): ciclo 8

- Instrução de ER 6 continua em execução.



| ER | Busy | OP   | Qj | Qk | Vj  | Vk   |
|----|------|------|----|----|-----|------|
| 1  | 0    |      |    |    |     |      |
| 2  | 0    |      |    |    |     |      |
| 3  | 0    |      |    |    |     |      |
| 4  | 1    |      |    |    |     |      |
| 5  | 0    |      |    |    |     |      |
| 6  | 1    | fdiv | 0  | 0  | 5.1 | -2.6 |

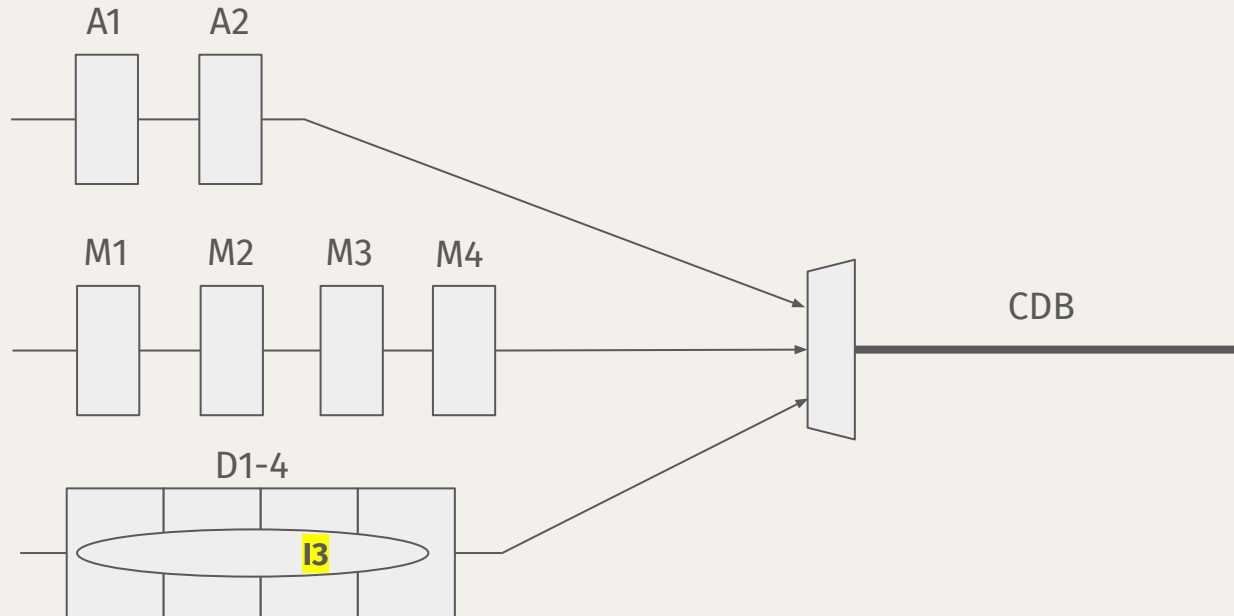
| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 0  | 0  | 0  | 0  | 6  | 0  |

| ER | Busy | OP   | Qj | Qk | Vj  | Vk   |
|----|------|------|----|----|-----|------|
| 1  | 0    |      |    |    |     |      |
| 2  | 0    |      |    |    |     |      |
| 3  | 0    |      |    |    |     |      |
| 4  | 1    |      |    |    |     |      |
| 5  | 0    |      |    |    |     |      |
| 6  | 1    | fdiv | 0  | 0  | 5.1 | -2.6 |

| REG | F1 | F2 | F3 | F4 | F5 | F6 |
|-----|----|----|----|----|----|----|
| Qi  | 0  | 0  | 0  | 0  | 6  | 0  |

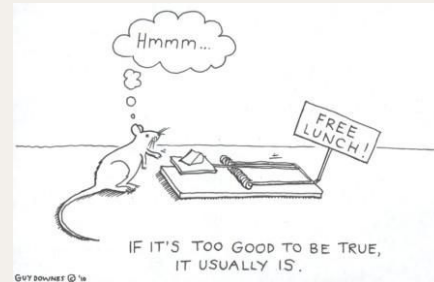
# Registos do CPU e unidades funcionais (fim do ciclo 8)

| REG   | F1    | F2   | F3   | F4  | F5    | F6  |
|-------|-------|------|------|-----|-------|-----|
| valor | 18.13 | 11.3 | -2.6 | 4.9 | -1.96 | 6.4 |



# Exemplo: observações finais

- Após 7 ciclos terminaram três das quatro instruções.
- A terceira instrução (fdiv) ainda está em execução após o ciclo 8
  - Mas a quarta instrução já terminou.
- A segunda instrução também terminou antes da primeira.
- As estações de reserva fornecem registos (internos) adicionais.
- A maior flexibilidade desta organização permite obter bastante melhor desempenho que a organização em *pipeline* única.
  - Mas aumenta muito a complexidade da implementação!



# Mais informações

- Secção 4.11, de David A. Patterson & John L. Hennessy  
“Computer Organization and Design” RISC-V edition 2nd ed. Elsevier.
- Secções 3.1, 3.4 e 3.5 de John L. Hennessy & David A. Patterson  
“Computer Architecture: A Quantitative Approach” 6th ed. Morgan Kaufmann.